CSE:598 Agentic AI

Phase 2 Report

Multi-Agent Method Proposal (τ-bench)

Sanyam Jain(1233149296), Abraham Lozano Serna (1221724634), Nihaarika Agarwal(1233370023), Omkar Korate(1230436430)

1. Introduction

In Phase 2, we diagnose why baseline tools using agents fail in τ-bench across Airline and Retail domains. We propose a multi agent framework designed to address the dominant observed failure modes.

2. Experimental Context

We analyze trajectories from Phase 1 for the following settings:
• Domains: Airline, Retail
• Baselines: ReAct, ACT, Function Calling
• Tool Agent Models: Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-32B
• Multiple trials per configuration

We analyze the failed trajectories and their errors and design a multi agent framework to address these failures.

3. Custom Error Taxonomy

- E1: Goal Incomplete / Premature Termination: Agent ends before fully satisfying the user request (missing final confirmation or required action).
- E2: Incorrect Tool Selection: Agent calls the wrong tool/API for the intended operation.
- E3: Incorrect Tool Arguments / Schema Error: Correct tool chosen but arguments are missing, malformed, or incorrect (IDs, fields, formats).
- E4: State Tracking / Memory Failure: Agent forgets or contradicts previously provided information across turns.
- E5: Policy / Constraint Violation: Agent violates domain constraints (eligibility windows, refund/exchange rules, restricted actions).
- E6: Clarification Failure: Agent fails to request required information or asks irrelevant questions that derail progress.
- E7: Tool Outcome Misinterpretation: Agent misreads tool output (treats error as success, ignores 'not found', etc.).

- E8: Sequencing / Planning Error: Agent performs steps in wrong order or skips required intermediate steps.

4. Proposed Multi-Agent Method: Plan–Execute–Verify (PEV)

Based on observed failure modes, we propose a three-agent system that separates planning, tool execution, and verification. Agents share a structured state object updated after each tool call.

4.1 Architecture Diagram

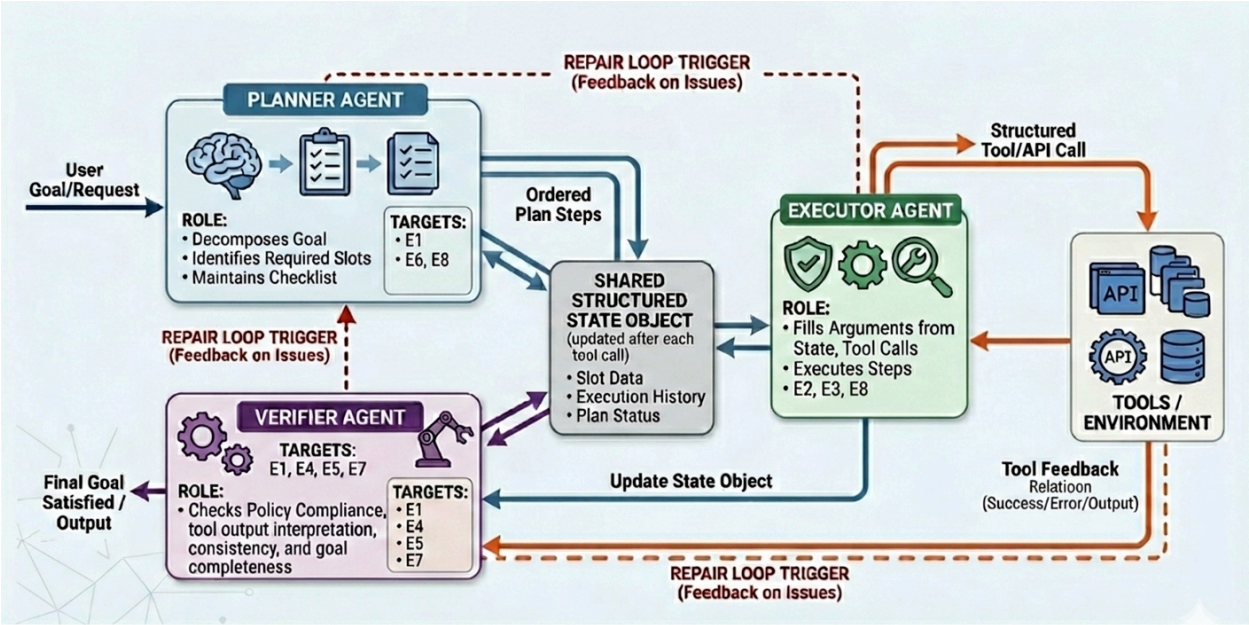

Figure: PEV multi-agent system (Planner + Executor + Verifier) with shared state and tool feedback.

With respect to the observed failure types, the proposed PEV multi-agent system differentiates between planning, tool execution, and verification. The system is structured around a shared structured state object that is maintained and updated after each tool call. The coordination cycle starts with the Planner agent, which produces ordered steps and identifies required slots. The Planner agent deals with error types E1, E6, and E8. The ordered plan is then sent to the Executor agent. The Executor agent creates structured tool calls, inserts arguments from the shared state object, and executes the plan. The Executor agent deals with error types E2, E3, and E8. Once the tool is executed, the Executor agent updates the shared state object. The Verifier agent checks policy compliance, tool output interpretation, consistency, and completeness in the shared state object. The Verifier agent triggers a repair cycle if errors are detected and deals with error types E1, E4, E5, and E7.

4.2 Agent Roles and Error Targets

- Planner: Decomposes user goal into steps; identifies required slots; maintains completion checklist. It targets the error types E1, E6, E8

- Executor: It generates structured tool calls; fills arguments from state; executes plan steps. It targets the error E2, E3, E8
- Verifier: It checks policy compliance, tool output interpretation, consistency, and goal completeness; triggers repair loop. It targets the error E1, E4, E5, E7

4.3 Pseudocode

```
---------------------------------------------------------------------------
Input: User Request (G), Domain Tools (T)
Output: Final Response or Task Termination
---------------------------------------------------------------------------
1: Initialize Shared State Object S ← {slots: [], history: [], status: null}
2: Plan ← Planner.GenerateDecomposition(G)
   /* Identify required slots and ordered execution steps */

3: while Goal G is not satisfied do:
4: curr_step ← Plan.GetNextAction()
5:
6: // Step 1: Execution
7: ToolCall ← Executor.GenerateStructuredCall(curr_step, S)
8: Result ← Execute(ToolCall, T)
9: S ← UpdateState(S, Result) // Update shared state with tool feedback
10:
11: // Step 2: Verification
12: Status ← Verifier.CheckCompliance(S, G)
   /* Check policy (E5), state consistency (E4), and output interpretation (E7) */
13:
14: if Status.has_issues() then:
15: Feedback ← Status.GetErrorDiagnostics()
16: Plan ← Planner.Repair(Feedback) // Trigger repair loop to target E1, E6, E8
17: else if Status.is_complete() then:
18: return GenerateFinalOutput(S)
19: end if
20: end while
---------------------------------------------------------------------------
```